# *Experiment No.: 8*

## *Interfacing Seven Segment LED using STM32F4xx*

*Name:* ____________________________

*Roll No.:* _________*Batch:* __________

*Date of Performance:* _________________

*Date of Assessment:* ___________________

| Particulars | Marks |
|---|---|
| Marks for Regularity (05) | |
| Marks for Performance (15) | |
| Understanding (Viva) (05) | |
| Total (25) | |
| Signature of Course Teacher | |

# Experiment: 8

**AIM:** Interfacing Seven Segment LED using STM32F407

**OBJECTIVE:**

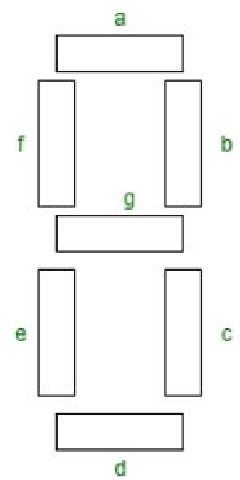
1. To understand the working of a seven-segment display (common anode / common cathode).
2. To interface a seven-segment display with STM32F407 GPIO pins.
3. To learn how to send binary patterns to display digits (0-9).

**APPARATUS:**

1. STM32F407 module development board.
2. Connecting wire.
3. Adapter.

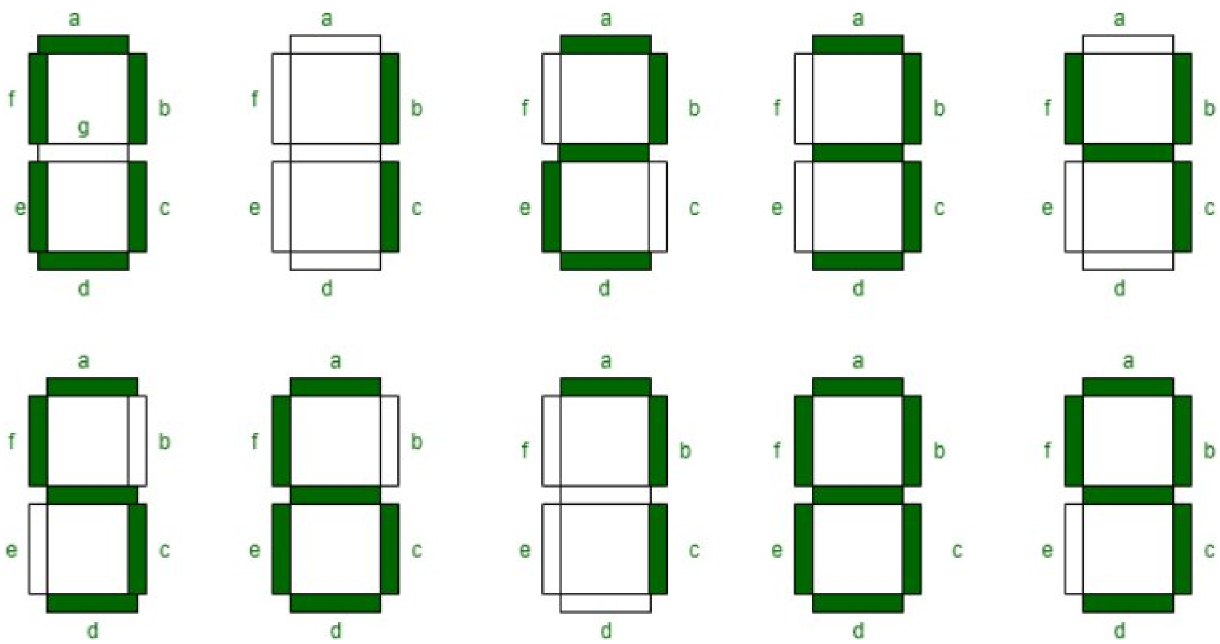
**THEORY:**

Seven segment displays are the output display device that provides a way to display information in the form of images or text or decimal numbers which is an alternative to the more complex dot matrix displays. It is widely used in digital clocks, basic calculators, electronic meters, and other electronic devices that display numerical information. It consists of seven segments of light-emitting diodes (LEDs) which are assembled like numerical 8.



**Working of Seven Segment Displays:**

The number 8 is displayed when the power is given to all the segments and if you disconnect the power for 'g', then it displays the number 0. In a seven-segment display, power (or voltage) at different pins can be applied at the same time, so we can form combinations of display numerical from 0 to 9. Since seven-segment displays cannot form alphabets like X and Z, so it cannot be used for the alphabet and they can be used only for displaying decimal numerical magnitudes. However, seven-segment displays can form alphabets A, B, C, D, E, and F, so they can also be used for representing each display unit is usually has a dot point (DP).




**Truth Table:** We can produce a truth table for each decimal digit.

Boolean expressions for each decimal digit that requires respective light-emitting diodes (LEDs) are ON or OFF. The number of segments used by digit: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9 are 6, 2, 5, 5, 4, 5, 6, 3, 7, and 6 respectively. Seven segment displays must be controlled by other external devices where different types of microcontrollers are useful to communicate with these external devices, like switches, keypads, and memory.

| Decimal Digit | Individual Segments Illuminated | | | | | | |
|---|---|---|---|---|---|---|---|
| | a | b | c | d | e | f | g |
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 2 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| 3 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 4 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 5 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 6 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 7 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 8 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 9 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |

**Types of Seven Segment Displays:**

According to the type of application, there are two types of configurations of seven-segment displays: common anode display and common cathode display.

- In common cathode seven segment displays, all the cathode connections of LED segments are connected together to logic 0 or ground. We use logic 1 through a current limiting resistor to forward bias the individual anode terminals a to g.
- Whereas all the anode connections of the LED segments are connected together to logic 1 in a common anode seven segment display. We use logic 0 through a current limiting resistor to the cathode of a particular segment a to g.

Common anode seven segment displays are more popular than cathode seven segment displays because logic circuits can sink more current than they can source and it is the same as connecting LEDs in reverse.

**Advantages of Seven Segment Displays:**

- **Simplicity:** Seven Section Presentations are straightforward and simple to use since they just showcase mathematical digits (0-9) and a couple of characters like A-F for hexadecimal numbers.
- **Cost-viable:** Seven Section Presentations are generally modest and require less parts to work than different sorts of showcases like LCDs or OLEDs.
- **High perceivability:** Seven Portion Presentations have high perceivability even in low light circumstances as they are intended to emanate splendid, high-contrast light in a particular example that is not difficult to peruse.
- **Durability:** Seven Section Presentations are strong and sturdy since they are produced using materials that are impervious to temperature changes and mechanical pressure.

**Common Cathode (CC) Configuration:**

In the common cathode configuration, all the cathodes (negative terminals) of the seven LEDs are connected to a common ground. Display segments are controlled independently by applying a HIGH (Logic 1) voltage to the anode of the particular LED. On applying the HIGH signal to an anode, current flows from the anode to the common cathode, and accordingly, the respective segment lights up.

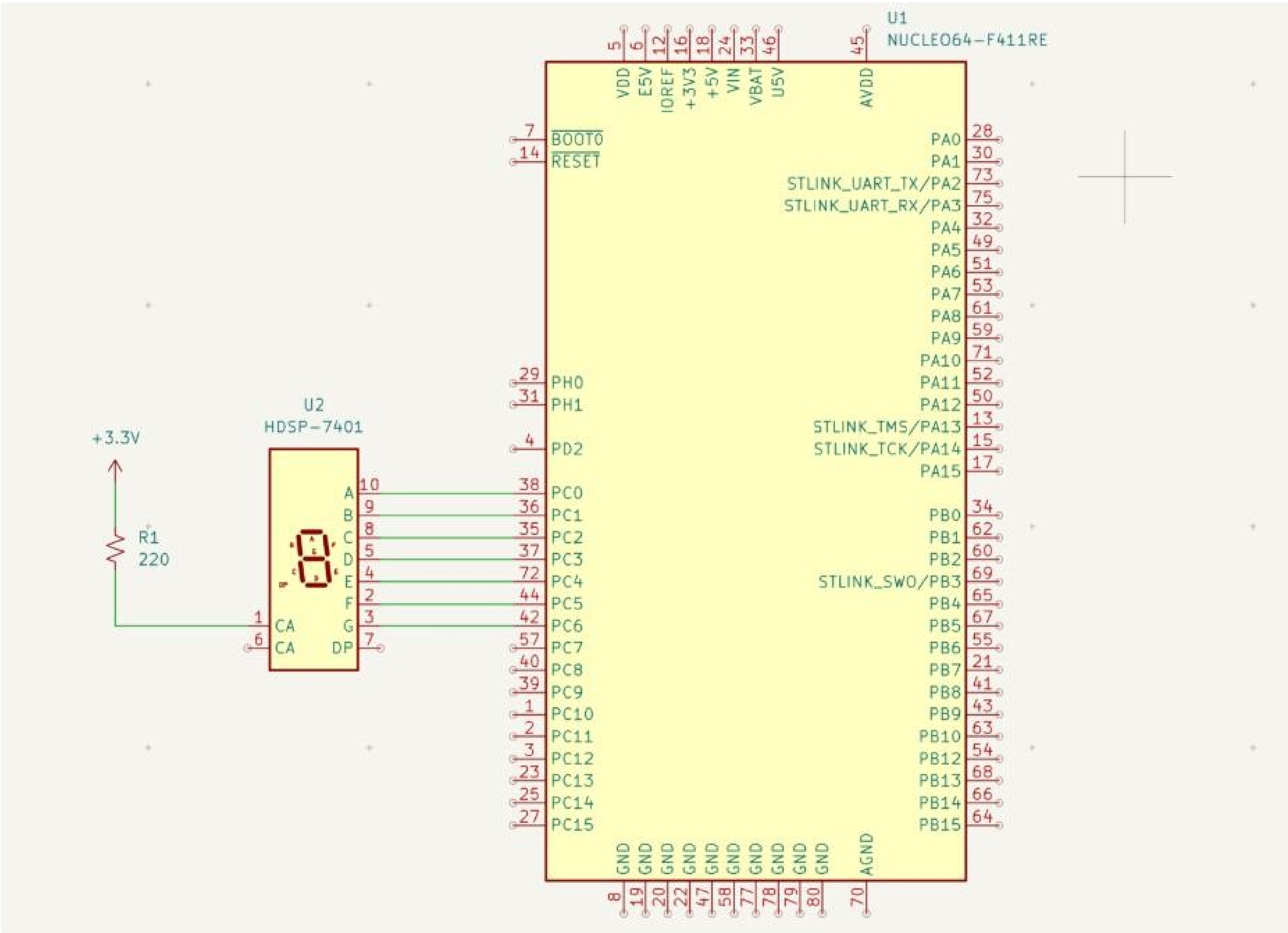
**Common Anode (CA) Configuration:**

In an anode common configuration, the positive terminals of the seven LEDs are connected together to a common positive voltage. Typically, this is VCC. Individual segments are turned on by grounding the respective cathode. A LOW, or logic 0, voltage at a cathode sends current flowing from the CA, through the LED segment, to ground and turns it on.

**REAL-WORLD APPLICATIONS:**

- Digital clocks
- Clock radios
- Calculators
- Wristwatches
- Speedometers

## INTERFACING DIAGRAM:




## LCD INTERFCAING DETAILS:

| Seven Segment | Pin Details | | |
|---|---|---|---|
| | STM32F407 | Processor Module | Description |
| A | PC0 | IO51 | Segment A |
| B | PC1 | IO52 | Segment B |
| C | PC2 | IO49 | Segment C |
| D | PC3 | IO50 | Segment D |
| E | PC4 | IO47 | Segment E |
| F | PC5 | IO48 | Segment F |
| G | PC6 | IO45 | Segment G |
| DP | PC7 | IO46 | Segment DP |

# 1. Using the STM32CubeIDE.

## 1.1. Creating/Choosing a Workspace.

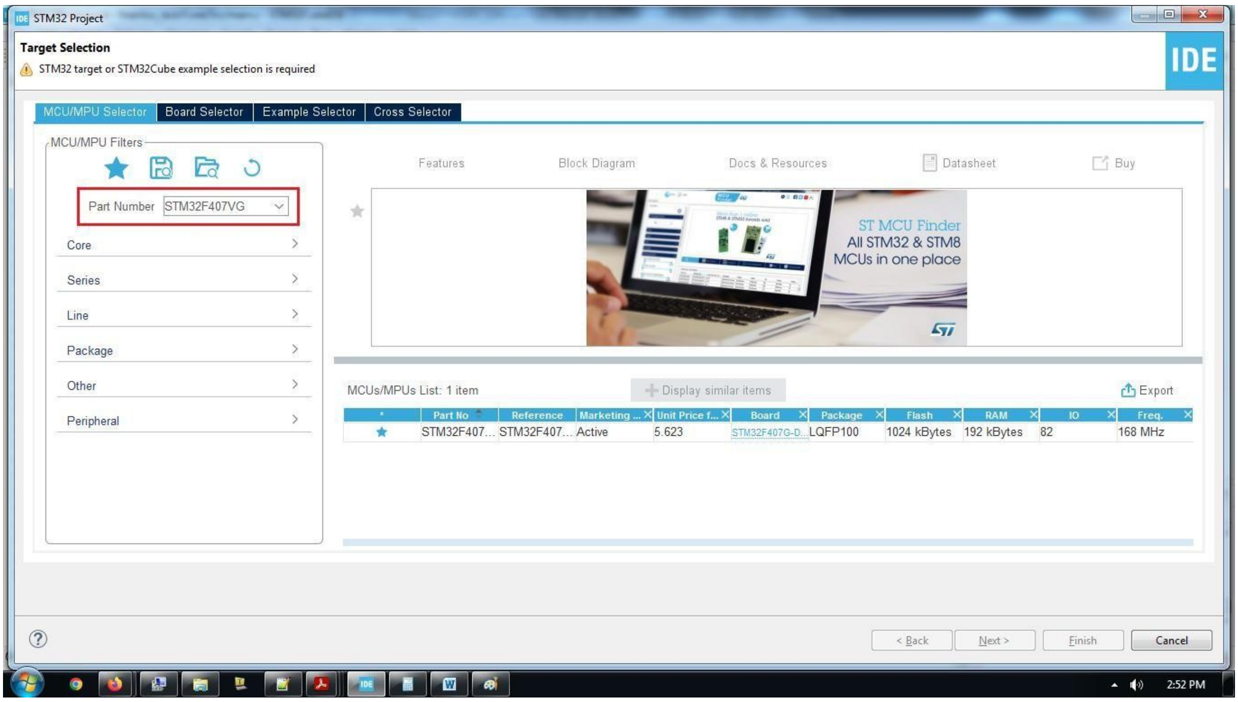
When you open the STM32CubeIDE for the first time, you will be directed to create a workspace for your projects. You can choose the default location for the project provided by the IDE (which will generally be located in the **users** folder in C: drive) or you can assign a different folder for the workspace. All your projects will be stored and referenced from this workspace folder. STM32CubeIDE is a eclipse based IDE and so you can have multiple projects in your projects window. Only one project can be active at one time. The Active project will get compiled or debugged or downloaded when you use the shortcut keys for the Compile, Debug or RUN actions.

## 1.2. Creating a New Project.

Open the STM32CubeIDE (preferably Run as administrator)

Step 1: To create a new project, click on File ☾ New ☾ STM32 Project.

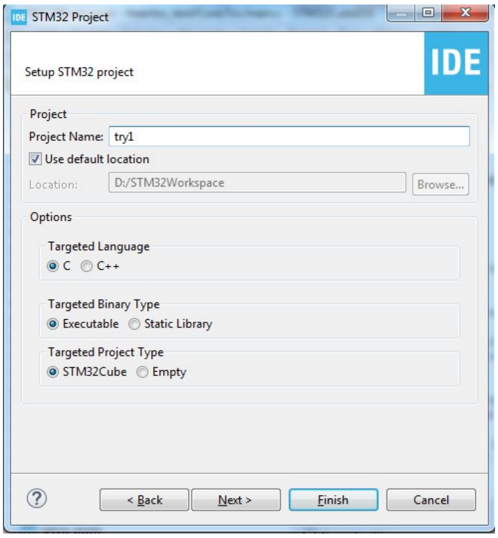

Step 2: Select the MCU part number.



In the Target Selection window, enter the part number **STM32F407VG** (this part is populated on the board), in the Part Number window.

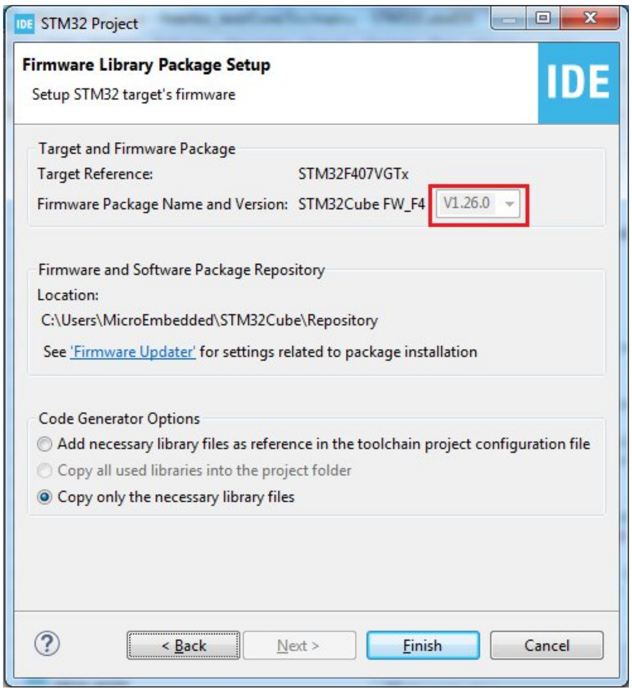
MCU/MPU Selector ☾ Part Number.

Now select the part (click near the star ★ ) and click Next. Step 3:



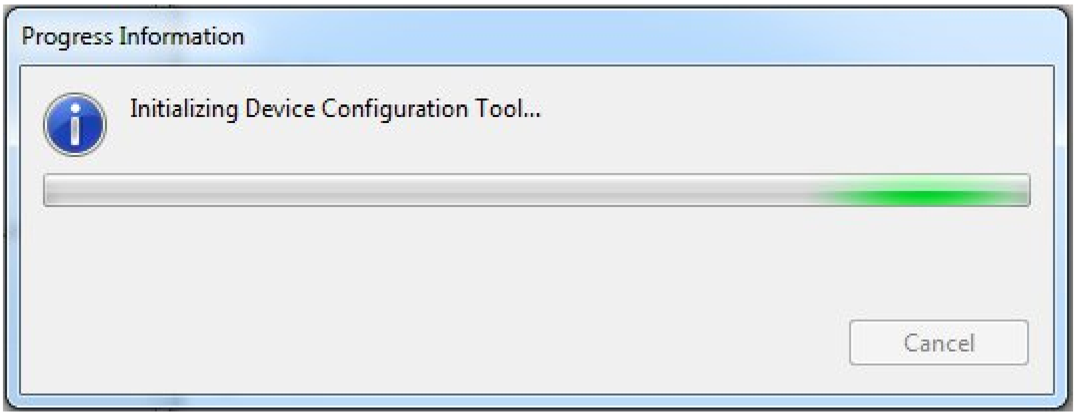

In the Project Name window, give a unique name to the project. (you cannot have two projects with the same name ). Click Next.

Step 4:

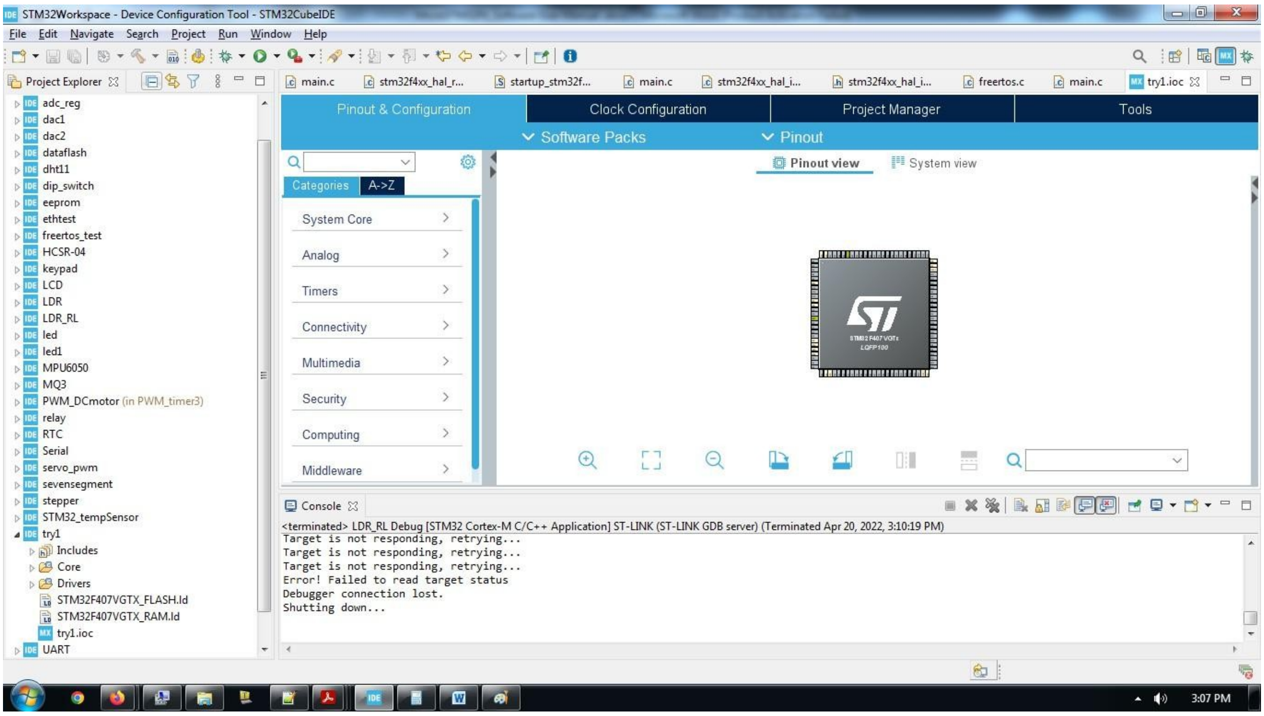



<u>Make sure that the Firmware Package Name and Version is **V1.26.0.**</u>

Click Finish to complete the process of creating the new project.




After the process is complete the project Template will get created.

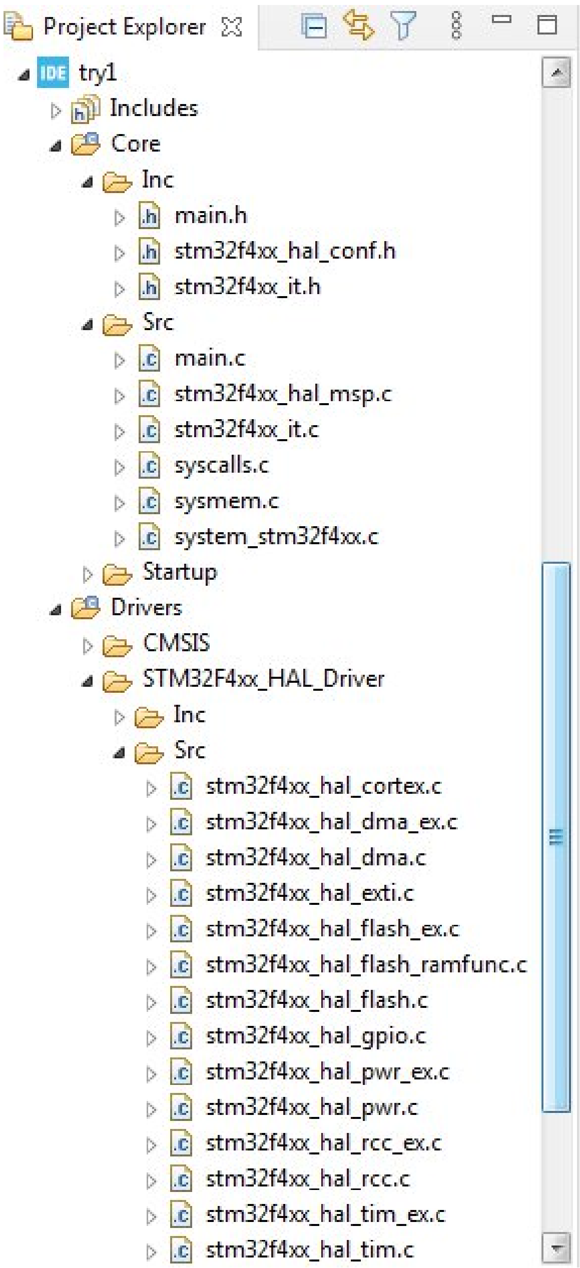



The Project Structure is as follows:
All the source code (.c) and header (.h) files for the HAL (library function for peripherals) are located in
C files :Project_name ☾ Drivers ☾ STM32F4xx_HAL_Driver ☾ Src
.h files :Project_name ☾ Drivers ☾ STM32F4xx_HAL_Driver ☾ Inc

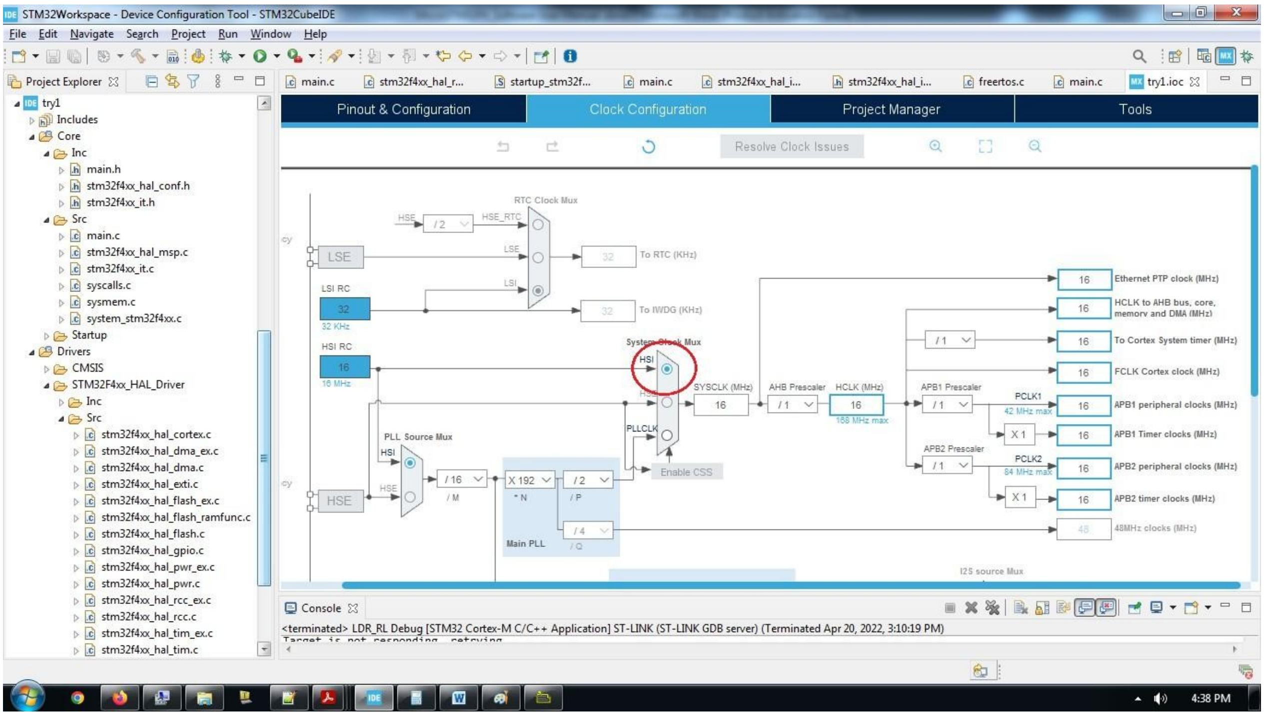
All the source code (.c) and header (.h) files for the user program are located in C files :Project_name ☾ Core ☾ Src
.h files :Project_name ☾ Core ☾ Inc

The user can write the program in main.c file.

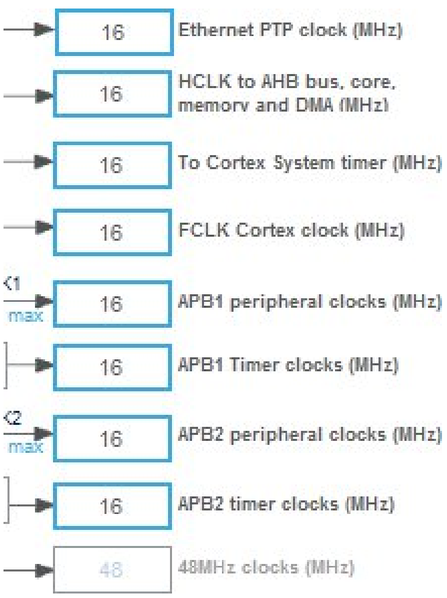
The code for clock initialization and HAL initialization are already done and available in main.c file.



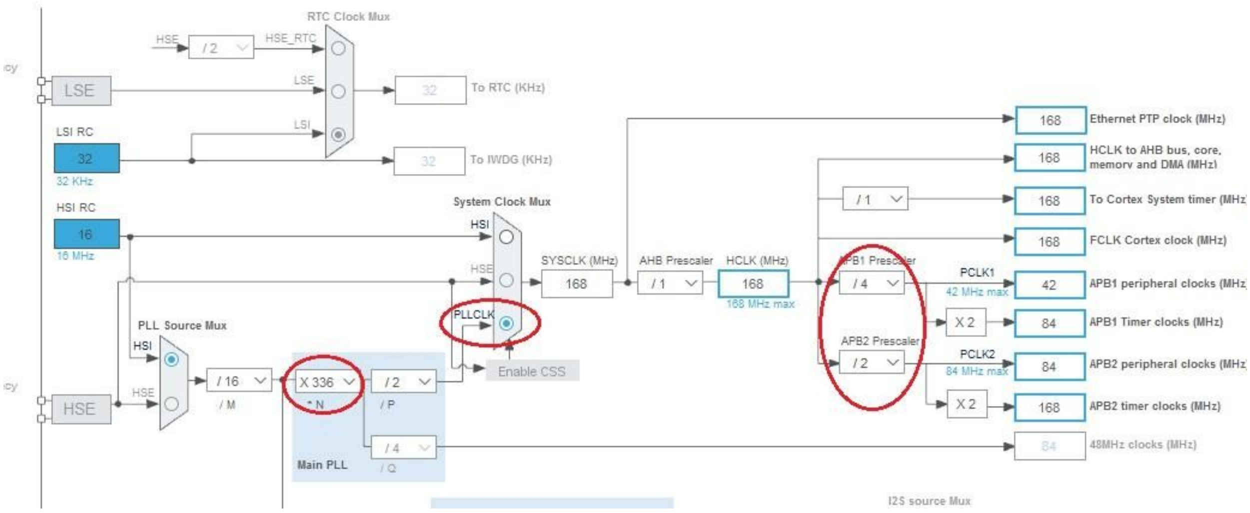
## 1.3. Setting the Clock Configuration.




Users can use the default clock settings for 16Mhz if they do not wish to make changes. Here we are using the (HSI) internal High Frequency RC clock to generate the clock signals for all the peripherals like AHB bus, APB bus etc.



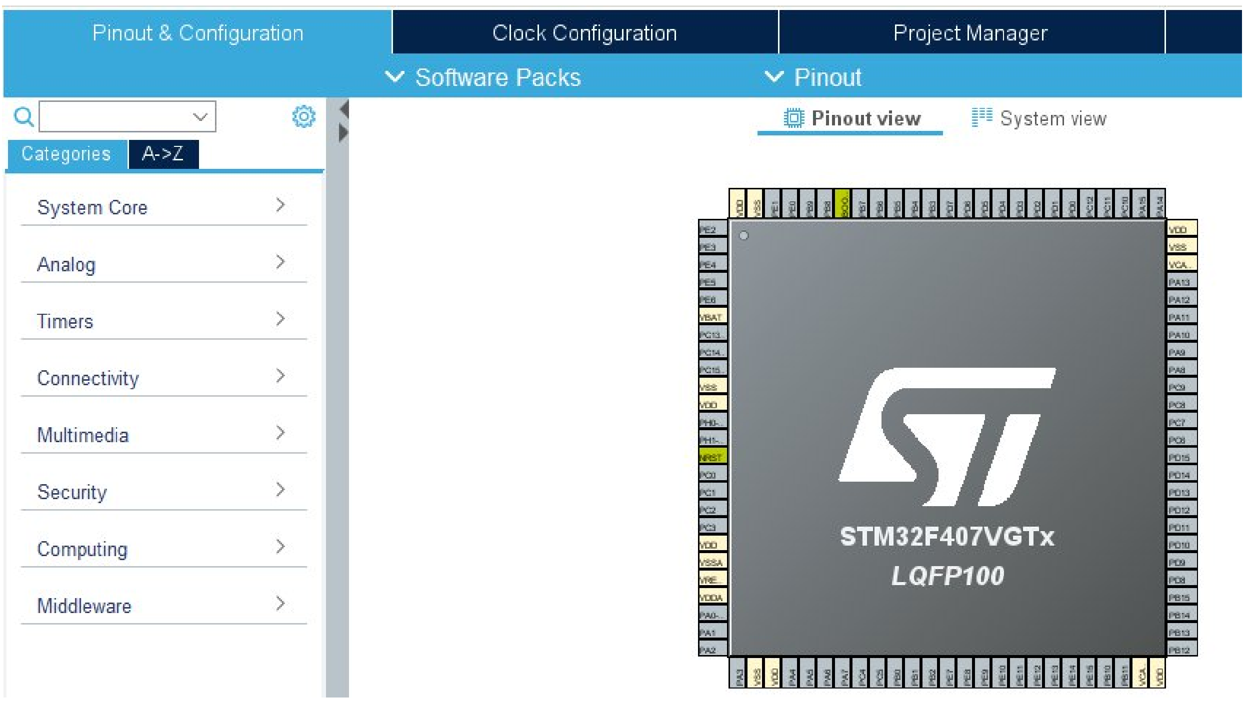
If the user wants the maximum clock frequency of 168 MHz, the PLL should be set as follows;




In the PLL section make N = 336
In System Clock Mux select PLLCLK In
APB1 Prescalar use /4
In APB2 Prescalar use /2

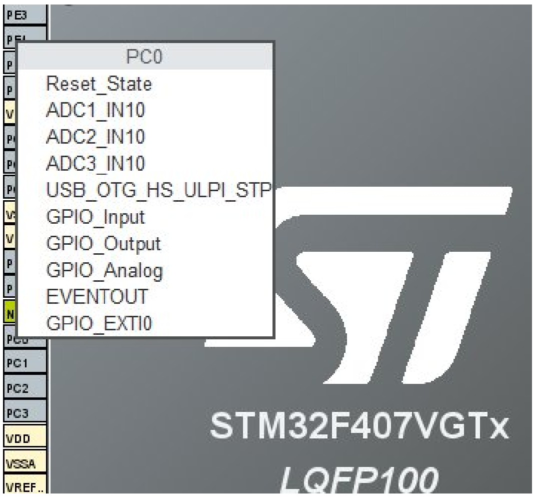
This will set all the clocks to the core and peripherals for maximum allowed frequency.

### 1.4. Pinout & Configuration.

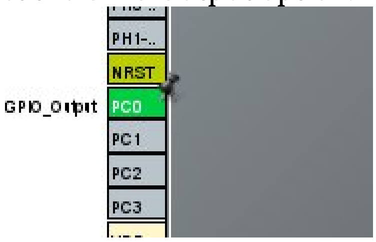


We can use the GUI tool in the IDE to set the pin configuration.

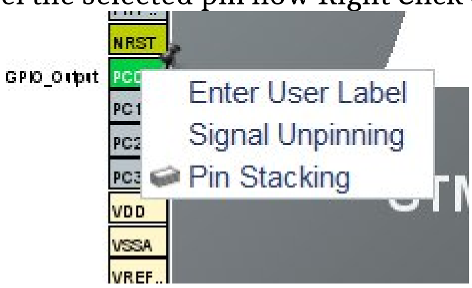
Select the pin (let's take pin PC0 which is connected to LED1). Locate the pin on the MCU pin map. Left Click on the pin PC0.



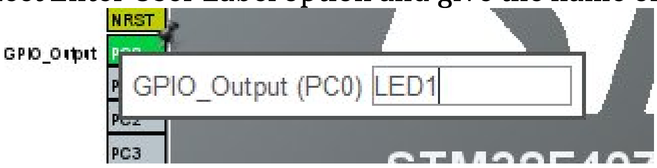

Select the GPIO Output option.




To label the selected pin now Right Click on the pin

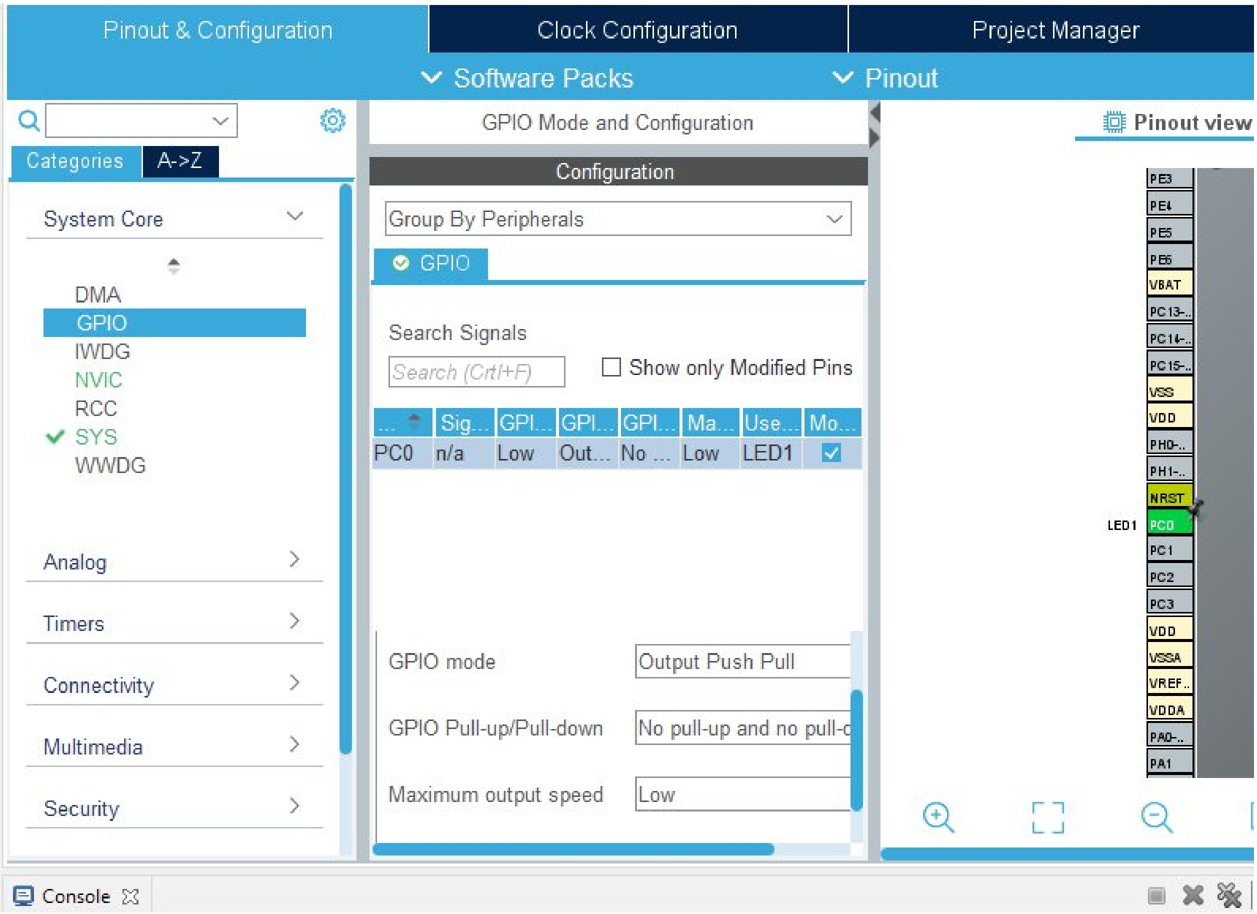



Select Enter User Label option and give the name of your choice in the window




Now when you press enter, the pin name will be set to LED1. This name will be used by the library and automatic code generation tool throughout the code.

You can also set the pin parameters from the tool.
Expand the System Core and select GPIO. The selected GPIO will be shown and below you can set the parameters for the selected Pin.



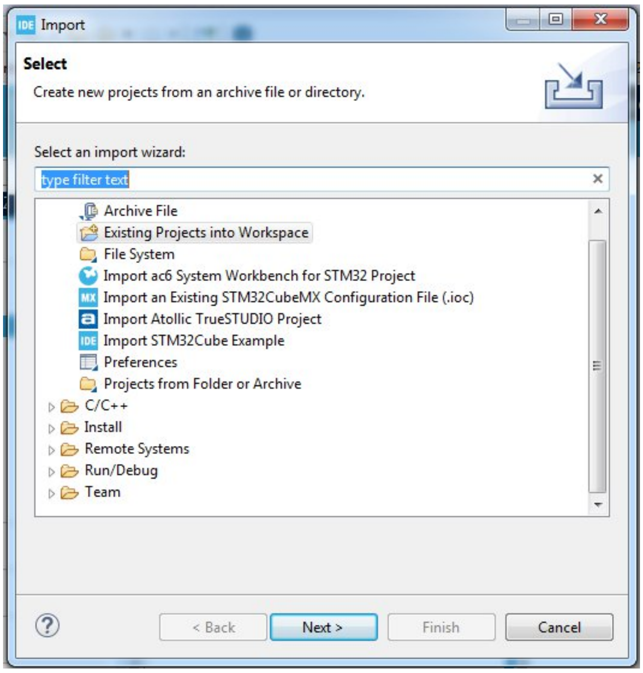

The same procedure applies to multiple pins and peripherals. To set the parameters for the peripherals, set the pins, select the peripherals from the Pinout &Configuration settings block and make appropriate changes.

After you have set the parameters, save the project. You will be prompted to generate code. When you click yes, the IDE will generate the initialization code and template in the main.c file.

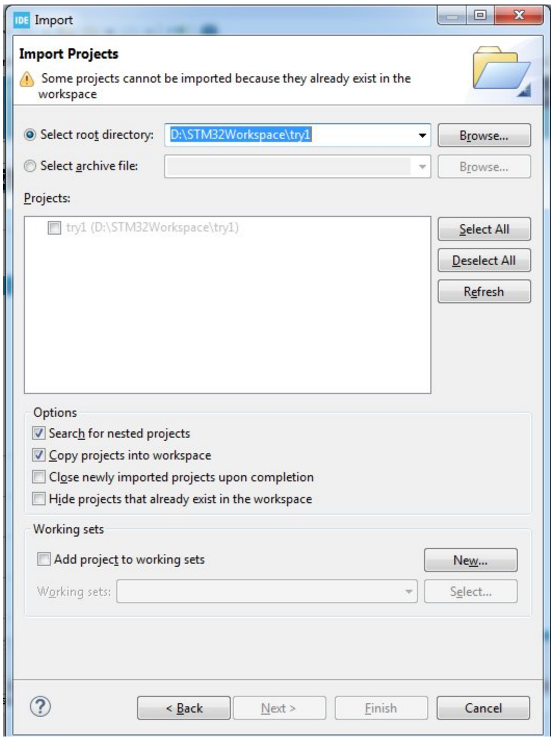
Alternatively, you can also click on the  Device Configuration Tool Code Generation to generate the code. Or go to Project ☾ Generate Code.

## 1.5. Importing an existing project in to the workspace.

When you want to import an existing project in to the workspace, click on File ☾ Import.




Select – Existing Projects into Workspace. Click Next.




In select root directory, browse to the folder which contains the project to be imported. Also make sure that the Copy projects into workspace field is selected in the options menu. Click Finish. The project will be imported into the workspace.

## 1.6. Building / Compiling the project.

After writing the code in the main.c file compile the project by Right clicking on the project name and click Build Project.

Also you can select the project name. click on Project ☾ Build Project.

Else you can select the project and click on  icon to build the project.

After successful build the bin file and elf file will be created. This file can be flashed into the microcontroller using the ST emulator.

## 1.7. Downloading into the Microcontroller.

To download the hex file into the microcontroller, connect the Emulator to PC using the USB cable and the FRC cable to connector CN22 (JTAG Header).
Power on the board.
Make sure that Boot0 and Boot1 switches are towards GND (2-3). And
switch SW13 is towards 2-3.

Now select the project, right click and select **Run As ☾ STM32 cortex m C/C++ Application**


Also, you can select the project ☾ Run ☾ Run. Else

click on  icon to download the file.

Press RESET switch SW38 and observe the output.

## 1.8. Debug the code.

To debug the code; select the project ☾ Right click ☾ Debug As ☾ STM32 Cortex-M C/C++ Application.

Also select the project ☾ Run ☾ Debug.

Or select the project ☾ click on debug icon. 

The hex fie will get loaded and the program will halt at main function. The IDE will enter the debug perspective and you can use the step into to step out to debug the code line by line.

**CONCLUSION:**